Fraud on a Quantum Budget

Variational quantum classifiers can learn to flag fraudulent card payments, but on a real quantum computer every gate adds noise and every training step costs circuit executions. Using the ULB Credit Card Fraud dataset, the task is to master the end-to-end VQC pipeline and then make it detect fraud as well as possible under noise, on a budget a quantum computer could afford.

OVERVIEW

Participants are challenged to build a Variational Quantum Classifier (VQC) that detects fraudulent credit card transactions on the ULB Credit Card Fraud dataset, and to make it hold up as well as possible under simulated quantum noise while keeping track of what its training would cost on real hardware. The challenge has two parts: a guided learning task that walks through a complete end-to-end pipeline, from the raw data to a VQC running under noise, and an open task in which each participant improves that pipeline under noise.

Fraud detection is a hard test for any classifier. Only 492 of the 284,807 transactions in the dataset are fraudulent (0.17%), so a model that never flags anything is 99.83% accurate and accuracy says nothing. What matters is how well a model ranks the rare frauds above the legitimate traffic, measured by the area under the precision-recall curve (AUPRC), and whether its decision threshold still catches fraud without drowning analysts in false alarms.

VQCs are the natural quantum candidate for this problem on near-term hardware: a circuit that encodes the data, followed by a trainable ansatz optimised in a hybrid quantum-classical loop [6]. Their weak points are noise and the cost of training. Every gate of a real device is slightly wrong and the errors add up: the 93-gate circuit of the guided notebook keeps its ranking of transactions at a depolarizing error of 1% per gate, but its threshold stops catching most frauds, and at 5% per gate the ranking itself degrades (a Spearman correlation of 0.65 between noisy and noiseless scores). Training is costly too: on a simulator the gradient comes for free, but on a quantum device every gradient entry needs extra circuit executions (two per parameter with the parameter-shift rule [7]) and every execution is repeated for hundreds or thousands of shots. Trained for 30 epochs on 590 balanced samples, the guided notebook's 4-qubit VQC would need 1,150,500 circuit executions on hardware, more than a billion shots at 1,024 shots each, and scoring the full validation set after every epoch would add another 1,708,860 executions.

The starting point is Dinuț et al. (2026) [1], which compared a quantum kernel SVM and a VQC against tuned classical models on this dataset at four qubits, under simulated depolarizing noise. Two of its findings shape this challenge. First, at four qubits the VQC

(AUPRC 0.494 ± 0.139 over 15 fits) stays below the tuned classical models (0.60 to 0.74), so a quantum result only means something next to a classical reference on the same inputs [22]. Second, under noise the VQC's AUPRC rose while its ranking of transactions decayed, so a single metric can report an improvement where the model is getting worse. A solution that works under noise must therefore be judged against a classical reference, over several seeds, and with more than one metric.

DOCS

Dataset

ULB Credit Card Fraud (Machine Learning Group, Université Libre de Bruxelles; Dal Pozzolo et al., 2015 [2]) contains the card transactions of European cardholders over two days in September 2013.

- Input: 30 numerical features. V1 to V28 are principal components released by the data owners to anonymise the transactions; Time (seconds since the first transaction in the dataset) and Amount are raw.
- Target: Class, 1 for fraud (492 transactions) and 0 for legitimate (284,315).
- Source: Kaggle (mlg-ulb/creditcardfraud) or OpenML dataset 42175, which is identical to it row by row [3]. The notebook downloads it automatically. Avoid OpenML dataset 1597: it has the same rows but no Time column.
- Training/Validation/Test Split: the guided notebook makes a stratified 60% / 20% / 20% split with seed 42, before any balancing, so that validation and test keep the real 0.17% fraud rate, and saves it as `split_indices.npz`. Task 2 uses this common split, so that results can be compared across participants.
- Preprocessing: Participants are allowed to resample, balance, subsample or reduce the training data and its dimensionality, provided every transformation is fitted on training data only and fully documented. Results must be reported on the test set of the common split, which keeps the original class distribution.

Task

The challenge has two parts. Task 1 is a guided learning stage; Task 2 is the competition, and only Task 2 is judged.

Task 1: a guided end-to-end VQC. Starting from the pipeline of Dinuț et al. [1], work through the provided notebook. It assumes the basics of quantum computing (qubits, gates, measurement) and no background in machine learning, PyTorch or PennyLane, and takes about four hours. It comes in two versions: one with short exercises, blanks to fill in that are each followed by an automatic check, and one with every solution and output. Quiz checkpoints along the way point to the accompanying quiz. The notebook covers, in order:

1. Machine-learning basics with PyTorch: features and labels, the binary cross-entropy loss, gradient descent, the Adam optimiser and the training loop.
2. Loading and exploring the dataset, and seeing why accuracy is meaningless at a 0.17% fraud rate.
3. Metrics for rare events: the confusion matrix, precision, recall, F1, MCC, ROC and precision-recall curves, and AUPRC as the primary metric.
4. Splitting into training, validation and test sets before any balancing, balancing the training set by undersampling, and preprocessing: scaling, PCA down to one feature per qubit, and mapping each feature to a rotation angle in $[0, \pi]$.
5. A PennyLane crash course: devices, QNodes, measurements, shots, backpropagation and the parameter-shift rule.
6. Building the VQC: a ZZ feature map followed by an EfficientSU2-style ansatz on 4 qubits with 32 trainable parameters, a readout from qubit 0, and the equivalent PennyLane templates.
7. Training with Adam on binary cross-entropy, keeping the epoch with the best validation AUPRC.
8. The cost of the same training on a quantum computer, in circuit executions and shots, counted by formula and with `qml.Tracker`.
9. Testing: AUPRC, AUROC, precision, recall, F1 and MCC at a threshold chosen on validation, against a logistic regression on the same four features, repeated over five seeds.
10. Noise: density matrices, the depolarizing channel and PennyLane noise models; the trained circuit under the challenge's noise model, its ranking of transactions (Spearman ρ between noisy and noiseless scores) and its threshold; and four tools for Task 2: noise-aware training, execution counting, SPSA gradients and zero-noise extrapolation.
11. The common split and the evaluation protocol of Task 2.

The goal is to understand what each step does and why, so that the same flow can be rebuilt and changed in Task 2.

Task 2: the best fraud detector under noise, on a quantum budget. Starting again from the original dataset, build your own end-to-end VQC pipeline, reusing anything you like from Task 1 or the article, and make its fraud detection hold up as well as possible under simulated depolarizing noise, while keeping track of what its training would cost on a quantum computer. Every submission must contain:

1. The baseline: the VQC of Task 1, evaluated with the protocol below over 5 seeds (the notebook's `evaluate_under_noise` does it).
2. At least one improvement aimed at noise, implemented and evaluated with the same protocol over at least 5 seeds, and compared with the baseline. Choosing the threshold on noisy validation scores is already part of the protocol, so it does not count as an improvement.
3. A classical reference trained on the same inputs as your circuit, at least a logistic regression.
4. A cost ledger for the baseline and for your method: the circuit executions and shots that training and validation would need on hardware, counted by formula or with `qml.Tracker`.

Evaluation protocol. Everybody is measured in the same way, as implemented in the notebook (Parts 10 and 11):

- Data: the common split. Balancing, encoding, architecture, epochs and threshold are fitted or chosen on training and validation data only; the test set is used only for final results.
- Noise: the circuit is decomposed into the gate set {RX, RY, RZ, Rot, H, CNOT, CZ}, and a depolarizing channel of probability p follows every gate on every qubit it acts on, simulated exactly with density matrices (`noisy_qnode` in the notebook), for p = 0.001, 0.01 and 0.05, plus the noiseless circuit.
- Threshold: the one that maximises F1 on validation scores computed under the same noise level.
- Metrics: test AUPRC as the primary metric, with headline values at p = 0.01 and p = 0.05; AUROC; Spearman ρ between noisy and noiseless test scores; precision, recall, F1 and MCC at the threshold. Report the mean and standard deviation over at least 5 seeds.

Possible directions, none of them mandatory, roughly from the easiest to the hardest:

- Fewer gates. Under gate noise, every gate costs accuracy, and two-qubit gates cost twice. Try fewer feature-map repetitions or ansatz layers, linear instead of full entanglement, merged single-qubit rotations, simpler encodings, fewer qubits, and removing the gates that cannot influence the readout (the notebook shows that 12 of the 32 parameters of the baseline never reach the measured qubit).
- Better readout and use of the data: other readout qubits or observables, class weights instead of undersampling, data re-uploading [19], and an initialisation that keeps gradients large [13, 14].
- Noise-aware training: train inside the noise model [16] and compare with training without noise and deploying under noise. Noise also flattens the gradients of deep circuits [15].
- Cheaper training: validate less often or on a smaller stratified subset, stop once a target validation AUPRC or loss is reached, use SPSA [11], shot-frugal or stochastic gradient estimates [8, 9, 10], or layerwise training [12]. Show how close the cheap model stays to the fully trained one, in AUPRC and in its ranking of the transactions.
- Error mitigation: zero-noise extrapolation [17], and what it costs in circuit executions.
- Further afield: classical surrogates of the trained circuit [20], or truncated simulation with Pauli propagation [21].

A method of your own weighs more than a known one applied as is, and an honestly measured negative result is a result.

Technical constraints. The quantum component must use **PennyLane**, and the classical component must use **PyTorch** (scikit-learn is allowed for preprocessing, classical baselines and metrics). The model has to stay a VQC: classical layers may prepare the inputs or rescale the circuit's output, but the classification must come from the quantum circuit. The circuit you report must be evaluated as a whole under the noise model; error mitigation is allowed if its extra executions appear in the cost ledger.

Resources

- The Task 1 notebook: `task1_guided_vqc_fraud_detection.ipynb`. It ran in Jupyter, VS Code or Google Colab and were tested with PennyLane 0.45, PyTorch 2.12 and scikit-learn 1.8. The whole notebook runs in about 10 to 15 minutes on a laptop CPU.
- Starting article: Dinuț et al. (2026) [1]. PennyLane documentation and demos [4, 5].

Note: in the provided pipeline the noiseless VQC reaches a test AUPRC of 0.58 ± 0.05 over five seeds, below a logistic regression trained on the same four features (0.70 ± 0.01). Keep a classical reference in Task 2 as well; a quantum result is read against it.

Use external resources freely but ensure your submission contains original contributions and uses citations where neccesary. AI coding assistants are allowed; they often write Qiskit code or call PennyLane functions that have moved or no longer exist, so check what they produce against the notebook and the PennyLane documentation.

References:

Starting point and data

[1] Dinuț, Constantinescu, Alexandrescu (2026). Comparative analysis and noise robustness study of quantum kernel methods and variational quantum classifiers for financial fraud detection. Electronics 15(11), 2489. https://doi.org/10.3390/electronics15112489

[2] Dal Pozzolo, Caelen, Johnson, Bontempi (2015). Calibrating probability with undersampling for unbalanced classification. IEEE SSCI 2015. The dataset paper, and why undersampling distorts probabilities. https://doi.org/10.1109/SSCI.2015.33

[3] The dataset on Kaggle and on OpenML (dataset 42175). https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

Tools and background

[4] PennyLane documentation. https://docs.pennylane.ai

[5] PennyLane demos on quantum machine learning. https://pennylane.ai/search?categories=quantum+machine+learning&sort=publication_date&contentType=DEMO

[6] Cerezo et al. (2021). Variational quantum algorithms. Nature Reviews Physics 3, 625. https://doi.org/10.1038/s42254-021-00348-9

[7] Schuld, Bergholm, Gogolin, Izaac, Killoran (2019). Evaluating analytic gradients on quantum hardware. Physical Review A 99, 032331. The parameter-shift rule. https://doi.org/10.1103/PhysRevA.99.032331

Cheaper training

[8] Kübler, Arrasmith, Cincio, Coles (2020). An adaptive optimizer for measurement-frugal variational algorithms (iCANS). Quantum 4, 263. https://doi.org/10.22331/q-2020-05-11-263

[9] Arrasmith, Cincio, Somma, Coles (2020). Operator sampling for shot-frugal optimization in variational algorithms (Rosalin). arXiv:2004.06252. https://arxiv.org/abs/2004.06252

[10] Sweke et al. (2020). Stochastic gradient descent for hybrid quantum-classical optimization. Quantum 4, 314. Gradients from few shots. https://doi.org/10.22331/q-2020-08-31-314

[11] Spall (1992). Multivariate stochastic approximation using a simultaneous perturbation gradient approximation (SPSA). IEEE Transactions on Automatic Control 37(3). https://doi.org/10.1109/9.119632

[12] Skolik, McClean, Mohseni, van der Smagt, Leib (2021). Layerwise learning for quantum neural networks. Quantum Machine Intelligence 3, 5. https://doi.org/10.1007/s42484-020-00036-4

Trainability, noise and mitigation

[13] McClean et al. (2018). Barren plateaus in quantum neural network training landscapes. Nature Communications 9, 4812. https://doi.org/10.1038/s41467-018-07090-4

[14] Grant, Wossnig, Ostaszewski, Benedetti (2019). An initialization strategy for addressing barren plateaus in parametrized quantum circuits. Quantum 3, 214. https://doi.org/10.22331/q-2019-12-09-214

[15] Wang et al. (2021). Noise-induced barren plateaus in variational quantum algorithms. Nature Communications 12, 6961. https://doi.org/10.1038/s41467-021-27045-6

[16] Wang et al. (2022). QuantumNAT: quantum noise-aware training with noise injection, quantization and normalization. DAC 2022. https://doi.org/10.1145/3489517.3530400

[17] Temme, Bravyi, Gambetta (2017). Error mitigation for short-depth quantum circuits. Physical Review Letters 119, 180509. Zero-noise extrapolation. https://doi.org/10.1103/PhysRevLett.119.180509

Architectures, simulation and benchmarking

[18] Havlíček et al. (2019). Supervised learning with quantum-enhanced feature spaces. Nature 567, 209. The ZZ feature map. https://doi.org/10.1038/s41586-019-0980-2

[19] Pérez-Salinas, Cervera-Lierta, Gil-Fuster, Latorre (2020). Data re-uploading for a universal quantum classifier. Quantum 4, 226. https://doi.org/10.22331/q-2020-02-06-226

[20] Schreiber, Eisert, Meyer (2023). Classical surrogates for quantum learning models. Physical Review Letters 131, 100803. https://doi.org/10.1103/PhysRevLett.131.100803

[21] Pauli propagation for the truncated simulation of quantum circuits (2025). arXiv:2505.21606. https://arxiv.org/abs/2505.21606

[22] Bowles, Ahmed, Schuld (2024). Better than classical? The subtle art of benchmarking quantum machine learning models. arXiv:2403.07059. https://arxiv.org/abs/2403.07059

RULES

Deliverables

Each submission must include:

1. **Code implementation of Task 2, runnable end to end with fixed seeds, from the dataset and the common split to the final tables**
2. **Presentation (.pdf or .pptx) describing:**
    - The end-to-end pipeline: preprocessing, balancing and encoding
    - Quantum circuit design and rationale, with its gate count and noise locations
    - The improvement: what it changes and why it should help under noise
    - Results with the evaluation protocol: baseline, your method and the classical reference at every noise level, as mean ± standard deviation over seeds
    - The cost ledger: circuit executions and shots for training and validation, against the baseline
    - What did not work, and what you would try next

! You must upload your presentation within the 24 competition hours. All presentations will be collected, and you will deliver your talk to the jury using the version you submitted.

Judging Criteria (in no particular order)

- Performance under noise: test AUPRC at $p = 0.01$ and $p = 0.05$, against the baseline and the classical reference
- Robustness: how well the ranking of transactions (Spearman ρ) and the threshold metrics hold up as noise grows
- Cost: circuit executions and shots for training and validation, against the baseline; reaching the same performance for less counts
- Quality and rationale of the circuit and of the method, including circuits that reduce qubit count or depth
- Trainability and stability across seeds
- Sound experiment design: the common split, no leakage, thresholds chosen on validation, a classical reference on the same inputs, results not cherry-picked
- Originality: a method of your own weighs more than a known one applied as is
- Clarity, technical cleanliness and reproducibility of the code
- Demonstration of research-oriented thinking: hypotheses, ablations, honest negative results
- Demonstration of good understanding of the problem and of the presented solution in detail, regardless of whether hand-coded or vibe-coded